[25-26_SEM1_CS341_B] - Web Technologies

Sprint 3

Mr. Kwadwo Osafo-Maafo

Team Members

Eldad Opare

Jerome Selorm Adedze

Marc Ettiene

Awindor

November 21, 2025

1. Project Overview

Project Name: ***Project Bonten***

Team: ***Group 12***

Stack: Apache (XAMPP/LAMP), PHP 8+, MySQL, HTML/CSS/JavaScript, Fetch API.

BONTEN is a web application built with HTML, CSS, JavaScript, PHP, and MySQL. This web application provides a platform where users can view and book events nationwide. The system supports user authentication, CRUD operations across major entities, and a responsive interface.

The architecture follows a three-layer structure:

1. Frontend (Client)
2. Backend (PHP Logic Layer)
3. Database (MySQL Relational Model)

Frontend Architecture

The frontend consists of:

- **Static pages**: Home, About Project, Team Profiles, Case Scenario.
- **Dynamic pages**: Login, Register, Dashboard, Record Management pages.

1. Implemented Functionalities

1. User Authentication

- Implemented user login and registration functionality (frontend & backend).
- Passwords stored securely (via hashing) and session management in place.
- Role differentiation (e.g., standard user and organizer view) has been defined or partially implemented.

2. Front-end Templates and Pages

- Basic site layout created (header, footer, navigation) using HTML, CSS (and possibly a CSS library).
- Core pages built:
    - Home / Landing page.
    - “About Project” page (describing purpose & scenario).
    - “Profiles” page for each team member (with picture, bio, role).
    - Case/Scenario page (describes the entity, problem, functions) and includes the initial storyboard.

2. Remaining Functionality

A) Fully functioning database

Sub-tasks

- Finalize ERD and ensure it matches implemented schema.
- Create and commit the database file containing CREATE TABLE statements with keys and constraints.
- Implement sample seed data for testing (users, sample items, sample orders/payments).
- Ensure foreign keys, indexes, and appropriate column types are in place.

B) Fully functioning backend (auth, payment, user handling)

I. Authentication (registration, login, logout, session)

Sub-tasks

- Implement registration endpoint (email, password, name). Validate inputs server-side.
- Implement login endpoint (email + password). Return session cookie or JWT.
- Implement logout (destroy session or revoke token).
- Add email uniqueness constraint and friendly error messages.

II. Payment processing

Sub-tasks

- Decide on a payment provider (e.g., Stripe, Paystack, MTN MoMo)
- Create a secure payment endpoint that initiates payment and returns a client token or a redirect URL.
- Implement a server webhook/callback endpoint to receive payment status and update the DB.
- Verify callback requests from the provider before updating the DB.
- Record payment transactions in a payments table.

III. User handling (profile management, roles, admin operations)

Sub-tasks

- Implement a profile update page and endpoint (name, email, and avatar are optional).
- Implement admin user listing, role-change, and deactivate/ban functionality.
- Add server-side validation for all profile forms.

C) Password hashing

Implementation details

- Create hash
- Store hash in a column.
- Use a password hash verifying function on login.